`publish_pipeline`

Scenario 1: a provider onboards a PDF. Nobody has asked anything yet.

```
step 1   ingest the PDF, extract passages ONCE
step 2   in parallel:
           2a  passages ──▶ Beckn catalogue metadata   (small; goes to the network)
           2b  passages ──▶ embeddings ──▶ vector DB   (large; stays with the provider)
step 3   /catalog/publish ──▶ the network layer
```

> **The manager document is `STEP2_WALKTHROUGH.md`** — a step-by-step runthrough of 2a and 2b with real values from a real run, and the full vector-DB technicality. Read that one. This file is orientation and how to run it.

In plain language first

Onboarding a bulletin is like cataloguing a library. A new bulletin arrives, and you do two unrelated things to it at the same time:

1. You write an **index card** — which crops it covers, which districts, which languages, how often it is refreshed. Small. That card goes out to the network so anyone can find you.
2. You put the **bulletin itself on a shelf** in your own building, arranged so you can find any paragraph inside it later. Nobody outside gets the bulletin.

Nobody has asked you anything yet. You do both because the bulletin was published, and you do it the same way regardless of who eventually asks or what they ask about.

The reason there are two branches at all — rather than one big index — is that the index card **cannot contain the bulletin**. A catalogue is cached by every consumer that fetched it, so advisory text placed inside it goes stale silently and there is no mechanism to recall it. Advice therefore has to be fetched live from the provider, which is exactly what branch 2b exists to serve.

One bulletin arrives with some of its pages **photocopied rather than typed** — a district name in real text, sitting above a table of crop advice that is actually a picture. Both jobs above see an empty page there, because neither job can read a picture. An optional step can retype those pages first, but only if what comes back genuinely reads as advice rather than as a smudge — otherwise the safer choice is to leave that page uncatalogued, same as before.

Run it

From a clean machine, start to finish:

```
git clone https://github.com/td1807/publish_pipeline.git
cd publish_pipeline

python3.11 -m venv .venv                      # any Python >= 3.10
.venv/bin/pip install -r requirements.txt     # ~3 GB: torch, then the e5 model on first run

.venv/bin/python main.py --all --fresh
```

Then the tests:

```
.venv/bin/pytest tests/test_v4.py -q -m "not semantic and not ocr"   # 54 tests, ~55s
.venv/bin/pytest tests/test_v4.py -q -m semantic                     # 1 test, needs the model
```

Publishing to a real node. Set `BECKN_NETWORK_NODE_URL` and the run POSTs instead of using the in-process stand-in. It retries transient failures — `PUBLISH_MAX_ATTEMPTS` (3), `PUBLISH_BACKOFF_SECONDS` (1.0, doubling), `PUBLISH_TIMEOUT_SECONDS` (60) — and records every publish to `.v4_state.json` so you can tell whether your coverage claims actually changed. See From here to production for what those two do.

Optional — recover pages that are pictures of tables. Off by default, because the run above and everything in `evidence/` is meant to be reproduced exactly as saved. The Rajasthan bulletin is 23 of its 30 pages that shape — a district heading in text above an advisory table that is a JPEG:

```
brew install tesseract tesseract-lang        # macOS. apt-get on Linux.
.venv/bin/pip install pytesseract

.venv/bin/python main.py --all --fresh --ocr
.venv/bin/pytest tests/test_v4.py -q -m ocr  # 5 tests, ~4 min — rasterises real pages
```

Run the plain command first, then the `--ocr` one, and diff the two — Karnataka and UP come back byte-identical; only Rajasthan changes (29 → 104 passages, 24.1% → 61.5% subject resolution). See "Known limits" below for why, and for the false-positive it took two attempts to gate correctly.

Three things about that first block are worth knowing, because each one produces a confusing error rather than an obvious one:

- **Python 3.10 is the floor, and macOS ships 3.9.** There is no `python` or `pip` on a stock macOS PATH at all, only `python3`, and that one is 3.9 — which is why `python …` and `pip …` fail with `command not found` before reaching any code here, and why `pip3 install` reports "no matching distribution" for packages that plainly exist. Calling the venv's own interpreter by path, as above, sidesteps all of it. `main.py` checks the version itself and says so plainly if it is too old.
- **Run `main.py`, not `-m publish_pipeline.run_scenario1`.** The module form still works, but only from the *parent* of the checkout, since the modules import each other relatively. `main.py` puts the parent on `sys.path` for you so the command works from inside the checkout. Either way, the directory must stay named `publish_pipeline`.
- **The first run is slow and needs the network.** Roughly 700 MB of torch at install time and ~2.2 GB of `intfloat/multilingual-e5-large` on first use. Both are cached afterwards (the model in `~/.cache/huggingface`), so later runs start immediately. To see the whole pipeline without either download, use `EMBEDDING_BACKEND=lexical`, which skips the model entirely and stamps `semantic=False` on every result so a degraded run cannot be mistaken for a real one.

Nothing else has to be running: Qdrant is embedded in the process against a local `.qdrant/` directory, there is no server, no Docker, and no port. That directory is not in git — `--fresh` rebuilds it. It is also locked while a run is in progress, so run one at a time; if a run is killed hard, delete `.qdrant/` and re-run with `--fresh`. Offline, prefix commands with `HF_HUB_OFFLINE=1` so sentence-transformers uses the cache instead of checking for a newer revision.

Saved output from a real run is checked in, so the walkthrough can be read without running anything:

Everything in `evidence/` is the **default run** — `main.py --all --fresh`, no OCR. That is what makes the reproducibility claim below checkable: clone, run that one command, and `git status` should report nothing changed except the `transactionId`, `messageId` and `timestamp` that are fresh per run. Output from `--ocr` is deliberately **not** checked in, so the two can never be confused for one another.

- `evidence/SCENARIO_1_TRANSCRIPT.txt`
- `evidence/publish_payload.json` — the actual `/catalog/publish` body (161 KB as published, 287 KB pretty-printed here)
- **`evidence/resources/` — one JSON file per bulletin**, each carrying that document's resources with their full `resourceAttributes`. Usually the more useful view: "what did *this* bulletin claim?" without scrolling past two other states.

file	state	resources	size
`karnataka.json`	IN-KA	71	170 KB
`up.json`	IN-UP	8	51 KB
`rajasthan.json`	IN-RJ	6	19 KB

Those are the **default run**, which is what `main.py --all --fresh` reproduces on any machine — no tesseract needed. Running with `--ocr` rewrites `rajasthan.json` to 12 resources and 45 KB; Karnataka and UP come back byte-identical. Don't commit that version over the default one.

- **`evidence/ocr/rajasthan.json` — the same bulletin with `--ocr`**, checked in separately so the difference is inspectable without installing tesseract. Diff it against `evidence/resources/rajasthan.json` and the `extraction` block states the case on its own:

`extraction`	default	`evidence/ocr/`
`passages`	29	**103**
`subjectResolution`	0.2414	**0.6117**
`districtsResolved`	41	41
`resourceCount`	6	**12**
crops in `agricultureSubjects`	8	**26**

Nothing regenerates this file — the default run writes `evidence/resources/` and never touches `evidence/ocr/` — so an `ocr`-marked test re-runs OCR and compares its resource ids, crop set, passage count and `subjectResolution` against a live run. That is what stops a hand-made artefact from quietly going stale.

- `evidence/message_update.reference.json` — the target shape, kept alongside so a test can diff against it
- `docs_pdf/` — this file and the walkthrough as PDF

To print a single resource's attributes straight to the terminal (matched on any substring of its id — note the order is `…-<category>-<area>`):

```
.venv/bin/python main.py --all --show-resource livestock-in-ka-koppal
```

It runs without the model, and says so

The default is the real `intfloat/multilingual-e5-large` (1024-d, multilingual). `EMBEDDING_BACKEND=lexical` runs with nothing downloaded, but it matches **spellings, not meanings** — no paraphrase, no cross-language — and stamps `semantic=False` on every result it returns. That is deliberate: a plausible-looking similarity score from a bag of character n-grams is the easiest way to mislead a room.

What one real run does

Three real government bulletins, chosen because they differ in ways that show up in the output — `--all --fresh`, on Apple Silicon (`device=mps`):

	Karnataka	Uttar Pradesh	Rajasthan	Rajasthan `--ocr`
pages	53	45	30	30
passages	241	240	29	**103**
chars per page in the text layer	1,745	2,326	448	—
language mix	`en` 240 / `hi` 1	`en` 100 / **`hi` 140**	`en` 2 / **`hi` 27**	`en` 2 / **`hi` 101**
subjects resolved	**83.4%**	56.7%	24.1%	**61.2%**
crops advertised	40	38	8	**26**
districts resolved	31 / 31	75 / 75	41 / 41	41 / 41
passages placed in a district	98.3%	32.1%	58.6%	55.3%
2a resources	**71**	8	6	**12**
capability types	4	4	4	4
2b vectors	241	240	29	**103**
max tokens / passage	254	299	314	314
2a time	~0.01 s	~0.01 s	~0.02 s	~0.01 s
2b time	15–186 s	22–154 s	4–49 s	12 s

Read the last column as the honest coverage of the Rajasthan bulletin, and the one before it as what the pipeline can see without OCR. 23 of that bulletin's 30 pages put a district heading in text above an advisory table that is a JPEG, so the default run is not measuring a thin bulletin — it is measuring a bulletin it cannot read. `--ocr` recovers 20 of the 24 pages the gate attempts and the file stops looking sparse: 8 crops become 26, subject resolution 24.1% → 61.2%, and district coverage stays at 41 / 41.

OCR is **off by default** because it needs the `tesseract` binary, which `pip` cannot install, and because OCR'd advisory text can carry corrupted pesticide doses (see Known limits). Everything in `evidence/` is the default run, so it reproduces on a machine without tesseract.

```
step 3   ACCEPTED — 3 catalogues, 85 resources, 164,720-byte payload
         network layer holds resourceAttributes only:
           85 resources · 51 subject URIs · 150 area codes · 13 topics
           · 4 subject categories · 7 weather parameters · 2 languages
         and zero advisory text — checked against real sentences from the source
         round-trip verified: every field held exactly as published

REFUSED  imd_karnataka_district_kannada.pdf
         has 0 of 1 pages with a text layer (0%, need 50%). This looks like
         a scan. Run OCR and re-ingest.

totals   3 onboarded · 1 refused · 0 failed · 510 passages · 85 resources · 510 vectors
```

The same command with `--ocr`, for comparison:

```
step 3   ACCEPTED — 3 catalogues, 91 resources, 181,733-byte payload
         141,206 bytes of resourceAttributes held
           91 resources · 51 subject URIs · 150 area codes · 13 topics

totals   3 onboarded · 1 refused · 584 passages · 91 resources · 584 vectors
```

Six more resources, all Rajasthan. The subject-URI and area-code counts do not move, because the crops OCR recovers were already advertised by Karnataka or UP — what changes is that **this** provider can now be found for them.

Why the three states differ so much is the interesting part, and it is real signal rather than a bug:

- **Karnataka publishes 71 resources from 53 pages** because it is organised as per-district advisory sections (`Agromet Advisory for Koppal district`), so almost every passage lands in a district and becomes part of a district-level capability.

- **UP publishes only 8 resources from *more* passages (240)** because it is organised by agro-climatic zone, and its tables list 7–8 districts per row. Those passages are honestly statewide, so they group into a handful of state-level resources — while still naming all 75 districts in `coverageAreas`, so a consumer can narrow down.
- **Rajasthan resolves only 24% of passages to a crop** because it genuinely is mostly weather warnings, not crop advisories. The run prints that as a warning rather than letting a thin catalogue look complete.

What OCR changes for the Rajasthan bulletin

Worth its own section, because it is the difference between "*this provider covers 8 crops*" and "*this provider covers 26*" — from the same file, on the same day, with no change to the vocabulary.

The Rajasthan bulletin is not a thin bulletin. It is a bulletin the pipeline cannot read: 23 of its 30 pages put a district heading in text above an advisory table that is a **JPEG**. Reading the text layer alone yields 448 characters per page against Karnataka's 1,745.

```
                                  default      --ocr
passages                               29        103
passages resolved to a crop          24.1%      61.2%
crops advertised                        8         26
districts advertised                41 / 41    41 / 41
resources in rajasthan.json             6         12
extractable characters             13,429     44,904
```

What lands in `evidence/resources/rajasthan.json`. The default run publishes four statewide resources plus two district ones:

```
crop-in-rj · horticulture-in-rj · livestock-in-rj · weather-in-rj
weather-in-rj-jodhpur · weather-in-rj-khairthal-tijara
```

`--ocr` adds eight more, all of them district-level advisory capabilities that the text layer never revealed:

```
crop-in-rj-bikaner · crop-in-rj-churu · crop-in-rj-jodhpur
crop-in-rj-pali · crop-in-rj-udaipur
horticulture-in-rj-udaipur · livestock-in-rj-udaipur · weather-in-rj-udaipur
```

That is the shape of the gain: a consumer app can now find this provider for *crop advice in Bikaner*, where before it could only find *weather advice for Rajasthan*.

The 18 crops recovered, every one of which was already in `crops.json`:

> Barley, Black gram, Buffalo, Cattle, Chilli, Cluster bean, Cotton, Goat, Green gram, Groundnut, Maize, Moth bean, Mustard, Paddy, Poultry, Sheep, Soybean, Sugarcane

Note `Buffalo`, `Cattle`, `Goat`, `Poultry` and `Sheep` — the entire livestock advisory of this bulletin was invisible without OCR.

The catch, and it is not small. 76 of those 103 passages carry `from_ocr: true`, meaning their text was transcribed from a picture and can contain corrupted pesticide doses — see the dose entry under Known limits before letting any of them reach a farmer. Branch 2a is unaffected: crop and district names are matched against a closed vocabulary, and dosages are never published.

Reproduce either side:

```
.venv/bin/python main.py --file imd_rajasthan_agromet.pdf --fresh           # default
.venv/bin/python main.py --file imd_rajasthan_agromet.pdf --fresh --ocr     # needs tesseract
```

`evidence/` holds the default run, so it reproduces on a machine without tesseract. The `--ocr` output for this bulletin is checked in separately as `evidence/ocr/rajasthan.json`, kept honest by an `ocr`-marked test that compares it against a live OCR run.

And the retrieval works

```
query: "pigeon pea flowers dropping, what should I spray"
  0.892  imd_up_agromet.pdf        p.39   res-…-crop-in-up
         "Pigeon Pea (Flowering) At flowering, scout for Helicoverpa larvae…
          spray Emamectin benzoate 5 SG @ 200 g/ha…"
  0.892  imd_karnataka_agromet.pdf p.16   res-…-crop-in-ka-koppal
  0.851  imd_karnataka_agromet.pdf p.13   res-…-crop-in-ka-kalaburagi
```

The word "tur" never appears in either bulletin — they say "Redgram" and "Pigeon pea". A query using "tur" still finds it (there is a test for exactly that), which is what the 1024-d multilingual model is buying. A Hindi query (`धान की फसल में सिंचाई`) returns Hindi passages from the UP bulletin.

Files

file	what it is	read it for
`config.py`	every setting, and what each degrades to	"what do I need installed"
`taxonomy/vocab.py`	crop/district/topic vocabulary + alias resolution	**the only thing both branches share**
`taxonomy/ids.py`	resource-id and point-id rules	why a reissue updates instead of duplicating
`taxonomy/data/*.json`	the vocabulary itself	what we can and cannot recognise
`ingest/document_text.py`	file → pages	ingestion, the formats it reads, and when it refuses
`ingest/language.py`	script/language detection + encoding check	the multi-language story
`ingest/ocr.py`	optional: pages that are pictures of tables	**why Rajasthan's coverage was thin**
`ingest/passages.py`	pages → `Passage` (text **and** facets in one object)	**why 2a and 2b cannot drift**
`beckn/models.py`	pydantic mirrors of `beckn.yaml`	spec conformance
`beckn/resource_attributes.py`	facets → `resourceAttributes`	**the heart of 2a**
`beckn/catalog.py`	passages → resources → catalogue	capability typing
`beckn/envelope.py`	the `message_update.json` envelope	step 3's payload
`beckn/validate.py`	spec + coherence + no-prose checks	what we refuse to publish
`vectors/embeddings.py`	the model, its prefixes, its 512-token limit	**the vector DB: model, dims**
`vectors/store.py`	Qdrant collection, payload, filters, ids	**the vector DB: schema, filters**
`network_node.py`	stand-in for the network layer	**what it holds**, and the two-hop shape
`publish.py`	step 3	validate-then-deliver
`scenario1.py`	`onboard()`, `publish_all()`, branch timings	the orchestration
`run_scenario1.py`	runnable, narrated	just run it
`tests/test_v4.py`	49 tests	most claims above, as assertions
`tools/md2pdf.py`	optional doc → PDF renderer	regenerating `docs_pdf/`
`tools/vocab_gaps.py`	optional: terms the vocabulary missed	triaging a new bulletin

Nothing here imports from `pipeline.*`, from `pipeline_beckn_v2` or from `publish_pipeline_beckn_v3`. The whole flow reads end to end without following imports into another package.

The design decisions, in one place

1. **One extraction pass feeds both branches.** A `Passage` carries the text (2b's payload) *and* its resolved facets (2a's raw material), and both branches read the same `Passage.facets()` method. Two extractors would drift, and the failure mode is nasty: discovery routes a question to a resource whose stored passages carry a different filter, so the provider searches inside a resource it just advertised and finds nothing. `test_facet_parity` holds the line.
2. **The unit of publication is a capability, not a document.** One resource per *(primary coverage area × capability category)* — "crop advisory for Koppal" — not one per PDF and not one per advisory row. Resource ids derive from *(provider, domain, category, area)* and contain nothing about the bulletin issue, so next Friday's reissue **updates** the same resources instead of minting duplicates.

Because the unit is *(area × category)*, **`@type` is a function of the category, not of the document.** One agromet bulletin therefore publishes four different capability types, and each carries only the attributes its type is for: a crop resource has `agricultureSubjects` and no `weatherParameters`; a forecast resource the reverse. `validate.py` fails a payload where the two disagree.

1. **The catalogue carries metadata only.** No advisory text, not even a snippet. Size is the small reason; staleness is the real one. `validate.py` fails the payload if prose creeps in, and a test greps the real payload for actual sentences from the source bulletins.
2. **`resourceAttributes` is where all the domain work lives — and it is what the network layer holds.** Beckn core leaves it as an open JSON-LD object on purpose (`Attributes`: requires only `@context` and `@type`, `additionalProperties: true`). Everything agriculture-specific goes there: `agricultureSubjects[]` with taxonomy URIs, `coverageAreas[]` as governed codes, `languages[]`, `topics[]`, `weatherParameters[]`, `forecastHorizon`, `updateFrequency`, `geographicGranularity`, plus an `evidence` block.

Stated in the right order: **the network layer holds the full `resourceAttributes`; what it does not hold is document text.**

1. **Areas are governed codes, never invented coordinates.** A polygon nobody surveyed is a polygon we made up, and downstream nothing can tell it from a real boundary. States use real ISO-3166-2 (`IN-KA`). There is a test asserting the payload contains no `coordinates` key at all.
2. **Extraction is rules-first.** The alias table does the work: deterministic, free, auditable, and byte-identical on every re-run — which matters because a churning catalogue republishes for no reason. **No LLM is in the default path, and no code path may mint a subject URI** that the taxonomy did not; `validate.py` checks every `subjectId` against the vocabulary.
3. **Closed-vocabulary fields are validated by membership, not by eyeball.** A topic must *be* a topic from `capabilities.json`. This is strictly stronger than scanning those fields for prose — which matters, because `"Spraying"` is simultaneously a canonical topic name and a word any prose detector would flag.
4. **A document we cannot stand behind is refused, not guessed at.** The default run includes the scanned Kannada district bulletin for this reason — `1 refused` in the totals is the check working, not a failure, and a demo that only ever showed the happy path would never demonstrate it. Three cases reach the same answer: a file no extractor can open, a document with no usable text layer (a scan), and a bulletin from a state the vocabulary does not cover. All three raise `UnusableDocument`, all three print a `REFUSED` line naming the reason, and none of them stops the other documents in the run. The alternative is a resource claiming coverage it cannot serve.

Known limits — read before demoing

- **A page that is a picture of a table reads as an empty page, and OCR is off by default.** The pipeline reads a text layer. The scan gate asks whether a page has *any* text, which is the right question for a fully scanned file and the wrong one for a hybrid: **23 of the Rajasthan bulletin's 30 pages carry a district heading in text above an advisory table that is a JPEG.** Ten characters is enough to clear the gate, so the document publishes — thinly, and with the `Crop coverage claims for this state are thin` warning that points at the vocabulary rather than at the real cause. It is **448 extractable characters per page against Karnataka's 1,745 and UP's 2,326**.

`ingest/ocr.py` recovers those pages. Measured: **29 → 103 passages, 24.1% → 61.2% subject resolution, 6 → 12 resources, 8 → 26 crops**, district coverage held at 41 / 41, and 13,429 → 44,904 characters across the 20 of 24 attempted pages the gate kept. **Every one of those 26 crops was already in `crops.json`** — the vocabulary was never the limit here, the text simply never reached it. Karnataka and UP are unchanged to the passage: UP has a text layer on every page, and Karnataka's image pages are forecast grids the gate correctly rejects.

It stays opt-in (`OCR_ENABLED=1`, or `--ocr`) for two reasons. `tesseract` is a system binary `pip` cannot install, so a default that needed it would fail on a fresh clone. And OCR'd text carries the risk in the next entry.

- **An OCR'd dose can be wrong, and a wrong dose is a farm-level harm.** This is the one caveat to read before letting `--ocr` output reach anybody. Tesseract on Devanagari damages exactly the tokens that matter most in an advisory. Measured on page 9 of the Rajasthan bulletin:

`source क्विनालफॉस 25 EC (1 लीटर/हेक्टेयर) recovered गस 25 50 (। लीटर/हेक्टेयर)`

The chemical name is destroyed and `25 EC` has become `25 50`. A farmer acting on that sprays the wrong thing at the wrong strength.

The pipeline's answer is provenance, not confidence: every passage recovered this way carries **`from_ocr: true`** on the stored point *and on every `Hit` that `search()` returns*, so an answering layer can act on it without reaching past the search API — it was payload-only until a farmer-question test showed the flag never reached the caller. `from_ocr` is deliberately **absent from `facets()`** so it cannot leak into a published coverage claim.

A dose question makes the risk concrete. Asked "फूल गिरने की समस्या के लिए कौन सी दवा और कितनी मात्रा", the top hit is an OCR'd page whose text reads `25 मिली. प्रति .00 लीटर पानी` — the leading digit of **100 litres** is gone — alongside `इमिडाबलोप्रिड 200 प्रतिशत`, a concentration that cannot exist (the label is 17.8% SL). The retrieval is correct; the transcription is not:

`0.872 imd_rajasthan_agromet.pdf p.7 from_ocr=True ← verify before showing 0.843 imd_rajasthan_agromet.pdf p.12 from_ocr=False` **An answering layer built on this index must either withhold `from_ocr` passages or show them with a verify-against-source marker and the page citation — never hand a dose from one to a farmer as settled fact.** Branch 2a needs no such guard: it publishes crop names, district names and topics matched against a closed vocabulary, so OCR noise either resolves to something that exists or resolves to nothing, and dosages are never published at all.

- **A district section used to swallow the districts a passage named.** Turning OCR on first *lost* 9 districts — Rajasthan fell 41 → 32 and the network's area codes 150 → 141. The cause was not OCR. `extract()` sets a passage's primary area from the district section it is inside, and `also_covers` was `current_section[1:]`, so the districts the passage explicitly named were discarded. Page 16's heading is 74 characters of text above a JPEG table — under `MIN_PASSAGE_CHARS`, so without OCR it was dropped and no section was ever active in that bulletin. OCR made the page survive, the heading matched, and it stayed in scope for the remaining 14 pages, including the annexure that lists all 41 districts bilingually.

`also_covers` is now the section's other districts **plus** the ones the passage names, deduplicated, section order first. That is the same argument the case-3 comment already made. With it, `--ocr` holds 41/41. In the default run it adds 14 area codes across two Karnataka resources and introduces no new districts — a statewide crop-stage grid on page 6 that sits under a district heading now publishes the 13 districts it forecasts for, instead of none.

- **The OCR accept gate is semantic, because every statistical version of it failed.** The first gate kept any page that gained vocabulary terms. Karnataka's tail pages are rainfall-probability grids that OCR renders as `[very`

`(७४० LIKELY|` token soup, and that soup gained three terms — enough to be accepted, and enough to publish a **Bengal gram** claim assembled entirely from noise. Alphabetic ratio, symbol density and mean word length were then tried and **all three overlap between the two classes**; the garbled grids actually score *higher* on alphabetic ratio than the genuine advisories (0.62–0.81 vs 0.48–0.68).

What separates them is what the page is *about*: a page worth recovering gives **advice**, and advice names agronomic topics. Rajasthan's advisory pages name 4–6; Karnataka's grids name 0–1. The threshold is 2, and Karnataka's three image pages are now correctly kept out (`test_ocr_refuses_forecast_grids_rather_than_inventing_coverage`).

- **An OCR crash once wore the costume of a working safeguard.** An early version passed raw bytes to pytesseract, which raises `TypeError`. A broad `except` caught it and reported "*OCR ran on 24 pages and was REFUSED on all of them*" — a total failure that read exactly like a gate doing its job. `OcrReading` now counts `pages_errored` separately from pages it judged and rejected, because only one of those is a bug.
- **Parallelism buys nothing measurable, and the run says so.** The two branches genuinely execute concurrently in a thread pool, but 2a costs ~10 ms against 2b's tens of seconds, so wall clock is simply 2b's time — the measured saving is ~0 (ratio 2,325×–27,678× in the latest run). The reason to run them concurrently is architectural (neither branch blocks the other, and either can fail without corrupting the other), **not throughput.** A real speed-up would come from parallelising *within* 2b. `BranchTiming.summary()` prints the ratio rather than claiming a win.
- **2b's timing varies by more than 12× on identical input.** Karnataka measured 14.9 s, 33.5 s, 83.6 s and 186.2 s across four runs on the same laptop (at 155 passages, before the crop-boundary chunking; at 241 passages four runs today gave 16.0, 17.5, 17.9 and 18.4 s) — fastest on an idle machine, slowest with the GPU hot and contended straight after a test run. The table above gives ranges for that reason. Quote no single figure; measure on the target hardware, and expect a dedicated GPU to be both faster and far more consistent.
- **An alias must not be a whole word inside another crop's name.** `gram` was listed as an alias for Bengal gram, and India grows black gram, green gram, horse gram and red gram. Resolution is word-boundary aware, so `tur` inside `turmeric` was never a problem — but `gram` inside `black gram` was, and the Karnataka and UP catalogues advertised Bengal gram coverage from bulletins that never mention chickpea. That is the precise failure `crops.json` warns about at the top of the file: a wrong subject URI routes a farmer to a provider that cannot help, and nothing downstream can tell.

Removing the alias withdraws both false claims. A test now asserts the general rule — every crop name resolves to that crop and no other — with one documented exception, `fodder sorghum`, which also resolves to `sorghum` because it genuinely is sorghum.

- **Passages are cut on crop boundaries as well as on length.** An IMD advisory table runs one crop's advice into the next with no blank line between them, so splitting purely on size produced passages that were about two crops and therefore precisely about neither. `_split_on_crop_change()` starts a new passage at a line naming crops the current one does not share, provided what is already buffered can stand alone; a line naming no crop always continues the run, because table rows rarely repeat their crop.

A crop change can strand a tail too short to survive `MIN_PASSAGE_CHARS`, so a run under that length is folded back into its neighbour. That is not cosmetic: the Karnataka bulletin ends a rain-impact list with "lodging of Banana plant." — 24 characters, and the document's only mention of that crop. Without the fold it was dropped and Banana disappeared from the catalogue. Keeping two crops in one passage is a far smaller cost than losing a line.

Measured across the three bulletins, before this change against today: passages 357 → 510, passages carrying a resolved subject **208 → 344**, passages carrying more than one crop **133 → 68**, and passages placed in a district 243 → 331. Text coverage is unchanged at 99.5% (per bulletin: 99.5% / 99.7% / 97.2%). Nothing was lost from the catalogue: same crops, same topics, same area codes per bulletin, and resources 74 → 84 at the time of this change (85 today, after Rajasthan's district list was completed — see below). The *percentage* placed in a district falls for UP only because the denominator grew; that absolute count is 76 → 77.

What it did **not** clearly improve is retrieval. Measured over all three bulletins with 14 farmer questions in Hindi and English: **12 unchanged, 1 better, 1 worse** (MRR@5 0.655 → 0.643 when the ground truth demands one specific Hindi passage; see the walkthrough §4.9 for the second, looser scoring and why the absolute figure should not be

quoted). Fall armyworm in maize improved from rank 3 to 2; "wilt in bengal gram" fell out of the top 5, into a cluster of black-gram and green-gram passages scoring 0.797–0.807 — a near-tie shuffle, not a structural loss. **The demonstrated gain is in labelling, not ranking.**

An English question does **not** fail against this corpus, though an earlier version of this section claimed it did — that was a measurement error, not a finding. Scored against "did the farmer get correct advice", `okra yellow mosaic virus, what to spray` and `how to protect livestock during heavy rain` both return the right passage at rank 1. What they return is the *English* advice from another state's bulletin rather than the local Hindi passage, because nothing in a bare semantic query says which state the farmer is in. In the real flow that is discovery's job: the consumer picks a provider by area code, and the follow-up search is scoped with `resource_ids`.

- **PDF is what these bulletins arrive as, but not what the code is limited to.** Ingestion goes through MuPDF, which reads **PDF, DOCX, TXT, HTML, EPUB and XPS**; all of those reach the catalogue, and a DOCX bulletin is covered by a test. Nothing downstream of `ingest/` knows what the file was — it consumes pages of text with numbers on them — so format is owned by one module. What does *not* read is the legacy `.doc` binary, spreadsheets and archives; those are refused by name:

```
REFUSED old_bulletin.doc Refusing to publish a catalogue from an unreadable document: old_bulletin.doc could not be opened (...). PDF and DOCX are what these bulletins arrive as; TXT, HTML, EPUB and XPS also read. The legacy .doc binary format, spreadsheets and archives do not — convert to PDF or DOCX and re-ingest.
```

Reading a format and extracting it *well* are different questions. The passage rules key off the IMD bulletin layout — `Major crops | Stage | Pest/disease | Advisories` tables and district headings — not off the file format. A DOCX laid out that way extracts at least as well as the PDF. A spreadsheet of the same data would open and produce nonsense.

- **Rajasthan's district list is the official 41, not only what the bulletin names.** The file originally carried 32, authored from the bundled bulletin, and the run reported "32 distinct" — which read like a shortfall against the state's real district count but was the vocabulary's own size. Nine were missing (Rajsamand, plus the eight districts retained in the December 2024 reorganisation: Balotra, Beawar, Deeg, Didwana-Kuchaman, Khairthal-Tijara, Kotputli-Behror, Phalodi, Salumbar), and four more failed to match because the bulletin spells them differently from the alias list — `झुंझुनु` for `झुंझुनूं`, `सवाईमाधोपुर` unspaced, and two carrying the font defect described below.

The bulletin's annexure does list all 41 bilingually (`डीग / Deeg`, `Khairthal – Tijara`), so this was lost coverage, not absent data: Rajasthan now places **41 / 41** districts against 32 before, 150 area codes against 141, and one further resource. Karnataka and UP are unchanged.

- **Three states, and a fourth one is refused rather than guessed.** `_STATE_MARKERS` in `ingest/passages.py` knows Karnataka, Uttar Pradesh and Rajasthan, and `taxonomy/data/districts.json` carries 147 districts across those same three. A bulletin from anywhere else cannot be given an area code without inventing a coverage claim, so it is refused by name:

```
REFUSED kerala_prices.pdf Cannot determine which state kerala_prices.pdf covers. Add a marker to _STATE_MARKERS in ingest/passages.py rather than guessing — an area code is a claim about coverage.
```

Adding a state is a marker line plus that state's districts with their aliases and local-script spellings. The marker is a minute; the district vocabulary is the actual work, and it has to be right, because those codes are published as coverage.

- **`context.domain` is not in Beckn v2.** The spec's `Context` has no `domain` property — v2 replaced it with `schemaContext`. We emit it because `message_update.json` carries it. Context is not `additionalProperties: false`, so it is tolerated, but it is a house extension rather than spec.
- **`CatalogPublishAction` is marked `deprecated: true`** in `core-v2.0.0-lts`, even though `/catalog/publish` remains the documented publish path and the spec offers no replacement.
- **Two documents cited throughout are not in this repository.** `beckn.yaml` is the Beckn core spec (`core-v2.0.0-lts`), which the models in `beckn/models.py` mirror by hand rather than vendor — so conformance here is checked against those models, not machine-checked against the published schema. `message_update.json`

is the target payload this work was given to match; it is checked in as `evidence/message_update.reference.json`, and a test diffs the built envelope against it.

- **The schema URLs do not resolve.** `https://schemas.openagrinet.global/...` fails DNS today, and `OpenAgriNet/network-specs` contains only a 15-byte README. `@context` is required by the spec, so we emit the intended URL rather than substitute one that happens to resolve.
- **`provider.availableAt` is omitted.** Beckn's `Location` requires a `geo` geometry and we have no surveyed boundary for any state. Coverage is expressed as area codes inside `resourceAttributes` instead. The supplied `message_update.json` has a hand-drawn Karnataka bounding box; we do not reproduce that.
- **District codes are not LGD.** LGD is the correct national scheme and its codes are numeric; we do not have the master loaded, so districts are emitted under `OPENAGRI-DISTRICT` with a readable code we can vouch for. Swapping in LGD is a two-field change in `taxonomy/vocab.py`.
- **Devanagari in these files has an encoding defect, and the two files have different ones.** The fonts are proper Unicode (Nirmala UI, Mangal), but the producing tool mis-maps glyphs. The UP bulletin emits `प्रदि` where `प्रदेश` belongs. The Rajasthan bulletin has a worse and more systematic defect: `ब` and `ि` are swapped, and the i-matra is emitted at the position it is *drawn* — left of its consonant — rather than in logical order, so `सिरोही` arrives as `बसरोही` and `बैंगन` as `िबैंगन`. Because the words it corrupts are crop and district names, it cost real coverage: the catalogue advertised 4 crops for a bulletin that discusses 8.

`repair_devanagari()` in `ingest/language.py` undoes it, and `repair_encoding()` in `scenario1.py` decides whether to trust the result — it scores both versions against the crop and district vocabulary and keeps the repair only if more known terms match. Measured: **Rajasthan 34 → 46 terms, applied; UP 121 → 78, refused.** The same transform run over the UP bulletin would destroy it, which is exactly why the gate exists rather than a blanket "fix Devanagari" pass. A repair that fires says so on every run:

```
encoding fix applied — 12 more vocabulary term(s) now match (34 → 46)
```

Measured defect rate before repair: **0.7% of Devanagari words in UP, 0.1% in Rajasthan**. Embedding is robust to it; exact-term lookup on an affected word is not. The alias tables absorb the specific corrupted spellings that appear, and `check_devanagari_encoding()` reports density on every run so a genuinely legacy-font file would be caught.

- **Hindi vs Marathi is not distinguished.** Devanagari is shared between them (plus Nepali, Sanskrit, Konkani). We report `hi` and set `ambiguous=True` with the sibling list, rather than shipping a model to guess.
- **Embedded Qdrant ignores payload indexes** and evaluates filters by scanning. Results are correct; latency is not a production figure. `VectorIndex.describe()` says which mode it is in for exactly this reason.
- **No Beckn signatures.** `AuthorizationHeader`, `AckSignatureHeader` and `context.key` are not implemented. Discovery trusts whoever calls it, and scoping a vector search by `resource_ids` **does not authenticate** that the caller was ever given them — resource ids are public. This is the largest gap and whoever builds the production consumer leg must know it.
- **`network_node.py` is a model, not an implementation.** No registry lookup, no signature verification, no multi-provider fan-out; persistence is a dict.
- **Scenario 2 is not built.** The follow-up loop (step 5 of the pipeline diagram) is out of scope here — scenario 1 involves no question. What is proven is that branch 2b's index *can* answer one: `run_scenario1` ends with a retrieval smoke check, and `test_retrieval_can_be_scoped_to_advertised_resources` demonstrates the discovery→filter→answer path.

From here to production

Everything above describes what this repository *is*. This section is what it is **not yet**, ordered by what blocks what — written so the next person does not have to rediscover it.

The distinction worth holding onto: **the design decisions carry forward, the plumbing does not.** Nothing below asks for a rewrite. The metadata/text split, the single extraction pass, the refuse-rather-than-guess discipline and

the measured gates are all load-bearing and unchanged by scale. What changes is almost every piece of infrastructure underneath them.

1. Signatures and registry — the blocker, not a task

Without these the pipeline cannot join a real Beckn network at all, so nothing else on this list matters until they exist.

`AuthorizationHeader`, `AckSignatureHeader` and `context.key` are unimplemented. A publish asserts it comes from IMD and nothing proves it; an ack asserts it comes from the network and nothing proves that either. Scoping a vector search by `resource_ids` **does not authenticate** the caller — those ids are public.

Done looks like: every outbound message signed against a registry-held key, every inbound ack verified, and a consumer leg that authenticates before it scopes. `network_node.py` is a stand-in for the counterparty and would be replaced entirely, not extended.

2. Publish reliability — done on `production-changes`

`publish.py` used to send one `httpx.post` and hope: a transient 503 ended the run with no record of which catalogues had landed.

It now retries with doubling backoff (`PUBLISH_MAX_ATTEMPTS`, default 3) and carries the envelope's own `messageId` as an `Idempotency-Key` header, so a retry is recognisably the *same* publish rather than a second one. Retries are deliberately narrow: 429 and 5xx are transient and worth repeating, while a 4xx is the node's verdict on the payload and is raised on the first attempt — re-sending a catalogue the node has already rejected on its merits is noise. Every retry prints, because a run that quietly took three attempts and one that worked immediately are not the same run.

Still open: nothing queues a failed publish for later. If all attempts fail the run exits non-zero and a human decides.

3. A vector store more than one process can open

Embedded Qdrant takes an **exclusive lock** on `.qdrant/`, so ingestion and query cannot run at once — that is the `already accessed by another instance` error, not a bug. It also evaluates filters by scanning, which `VectorIndex.describe()` prints on every run precisely so its latency is never quoted as production.

Cheap part: `VectorIndex` already takes `url`, so pointing at a Qdrant server is a config change. *Real part:* running that server, and re-declaring the payload indexes so filters stop scanning.

4. A record of what was published — done on `production-changes`

`STATE_FILE` was declared in `config.py` and never read or written. It now holds a ledger: every publish appends its timestamp, `transactionId`, `messageId`, target, ack status, resource count, payload size and a `claimsHash`.

`claimsHash` is the load-bearing field, and what it *excludes* is the point. Every envelope carries a fresh `transactionId`, `messageId`, `timestamp` and validity window by design, so a hash over the raw envelope would differ on every run and answer nothing. The hash covers the catalogues with those removed — so republishing unchanged coverage hashes the same, and a hash that moves means **the claims moved**. That is the difference between a log and a diff, and the run says which it was:

```
ledger        0af0f350abac2fce — coverage claims unchanged since the previous publish
```

The ledger is written *after* the ack, never before — an attempt that never reached the node is not a claim anybody holds. A failure to write it prints and is not fatal: the catalogue is already out there, and failing the publish over a local file would be the larger lie.

Still open: no rollback. The ledger tells you *that* claims changed and lets you diff hashes; restoring a previous catalogue is still manual.

5. Batch processing and the model as a service

One document at a time, in one process. And e5-large is loaded per run, with **12× timing variance on identical input** — Karnataka measured 14.9 s, 33.5 s, 83.6 s and 186.2 s on the same laptop.

Partly addressed on `production-changes`: a crashing document no longer takes the batch with it. Only `UnusableDocument` was caught before, so any other exception ended the run and every document already ingested went unpublished — one malformed file cost the whole batch. Failures are now caught per document, counted **separately from refusals**, and the run continues:

```
documents onboarded   3
documents refused     1      ← a decision this pipeline made on purpose
documents failed      0      ← a document that should have worked and did not
```

Keeping those two counts apart matters: collapsing them would hide a bug behind a message that reads like a safeguard working, which is the mistake `ocr.py` records having already made with its errored-vs-refused page counts. The exit code follows the same logic — a refusal exits 0, a failure exits **1**, so cron, a queue or CI can tell without parsing stdout.

Done looks like: a queue with per-document retry so a crash costs one item, and embeddings behind a long-lived service. `EMBEDDING_BACKEND=remote` already exists for the second half — but note its `token_report()` returns `None`, because the server's tokenizer cannot be inspected, so the truncation check switches off exactly where it cannot be observed.

6. Reference data is the scaling wall, not throughput

Coverage is authored, not discovered: `_STATE_MARKERS` names three states in source, and `districts.json` carries their districts written by hand. Adding a state is a code change plus authored vocabulary, not configuration.

Rajasthan showed the cost concretely. It carried 32 districts for a long time — the ones the bundled bulletin happened to name — while the state has 41. The missing nine were published as no coverage at all, from a bulletin whose annexure lists every one of them bilingually.

Done looks like: districts loaded from **LGD**, the official national register, with its numeric codes replacing `OPENAGRI-DISTRICT`. Emitting them is the two-field change in `taxonomy/vocab.py` noted under Known limits; the work is loading and reconciling the master, and handling reorganisations like Rajasthan's 33 → 50 → 41.

Throughput is bought with machines. This is bought with authoritative data, and it is the thing that actually limits a national deployment.

7. Enforce the OCR dose guard before farmers see anything

The only item here with physical consequences, and the reason it is last is ordering, not priority — it gates *release*, not the work above it.

OCR damages the tokens that matter most. A dose question returns, as its **top hit**, text reading `25 मिली. प्रति .00 लीटर` — the leading digit of 100 litres is gone — and `इमिडाबलोप्रिड 200 प्रतिशत`, a concentration that cannot exist.

Every such passage carries `from_ocr` on the stored point *and* on every `Hit` that `search()` returns. But **nothing consumes it yet**, because the answering layer does not exist. Today the protection is a boolean and this paragraph.

Done looks like: enforcement somewhere it cannot be bypassed — an answering layer that withholds `from_ocr` dosages, or surfaces them only beside the page citation and a verify-against-source marker. A `search(exclude_ocr=True)` parameter was considered and deliberately not added: with no consumer to say whether withholding or flagging is correct, it would bake a policy into the API, and for cotton and livestock questions the only Rajasthan answer *is* an OCR'd one — excluding it returns another state's advice instead.

And the half that is not built

Scenario 2 — a farmer asks, the provider answers — is out of scope here, and no amount of the above substitutes for it. What is proven is that branch 2b's index *can* answer: the run ends with a retrieval smoke check, and `test_retrieval_can_be_scoped_to_advertised_resources` demonstrates the discovery → filter → answer path.

What already holds

Worth stating alongside the gaps, because it is what makes them safe to work against: **60 tests**, and every artefact in `evidence/` regenerates byte-for-byte from `main.py --all --fresh`. The documentation can be checked by running one command rather than by trusting it.